

* Objectives

→ Understand what is pointer

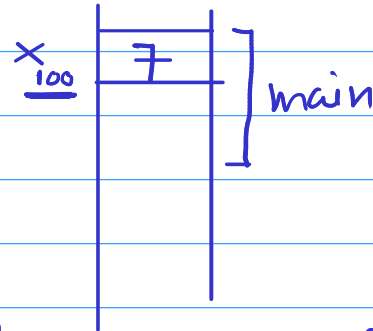
→ How to declare, initialize and access pointers and data being pointed.

→

```

int main() {
    int x = 7;
    printf("%d", x);
    // printf("%d", &x);
    // Above statement is
    // invalid. can't use %u
    printf("%p", &x); // 0x64

```



int 4 bytes
 0 to $2^32 - 1$
 (circled) %u
 $1024 = 2^{10} = 1 \text{ KB}$
 $2^{10} \times 2^{10} = 2^{20} = 1 \text{ MB}$
 $2^{30} = 1 \text{ GB}$
 $2^{32} = 4 \text{ GB}$

```

int i = 10;
// int p = &i;
// Above statement is
// invalid

```

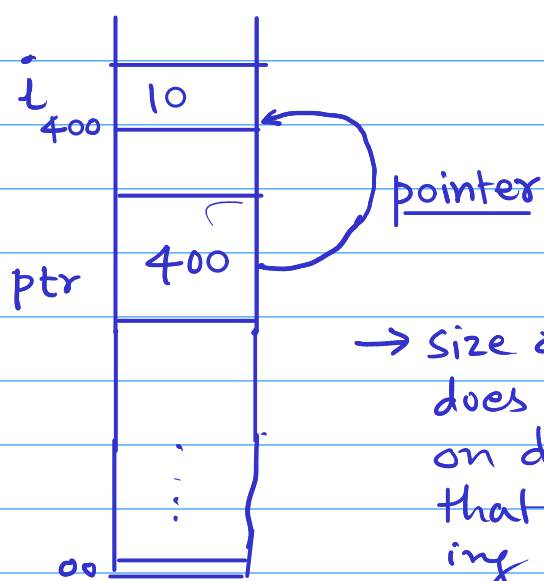
int *ptr = &i;
 Declaring pointer ptr and initializing it

{ int *ptr; // declaration
 ptr = &i; // initialized ptr with addr of i

X int *pt = 32; // Invalid

→ pointer can not be initialized (assigned) with constant value, except 0 or NULL.

✓ { int *pt2 = 0;
 int *pt3 = NULL; } Valid



→ Size of pointer does not depend on data type that it is pointing to.

→ On modern computers size of pointer is 8 bytes.

↳ Range?

0 to $2^{64} - 1$

2^4 Zetta bytes.

int i = 6;

int *ptr;

ptr = &i;

↑ Address of operation

⑥ printf("%d", i);

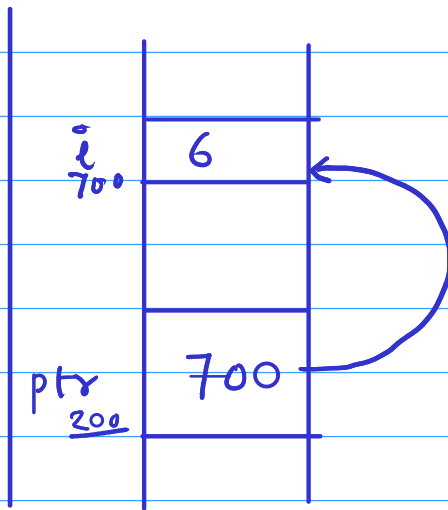
X printf("%d", ptr);

⑦ printf("%p", ptr);

⑥ printf("%d", *ptr);

dereferencing / indirection operation.

② printf("%p", &ptr);



i — int
ptr — int *
*ptr — int

int i = 3.7;

X int *ptr = 12;

int i = 12;

X int *ptr = i;

int *

int

H.W

int j;

int i = (int) &j;

```
int x = 23;
char c = 'p';
float f = 3.14;
```

```
// int *ptr1 = &c; // Invalid
```

↳ int * can only hold address of int variable

```
int *ptr1 = &x;
```

```
✓ char *ptr2 = &c;
```

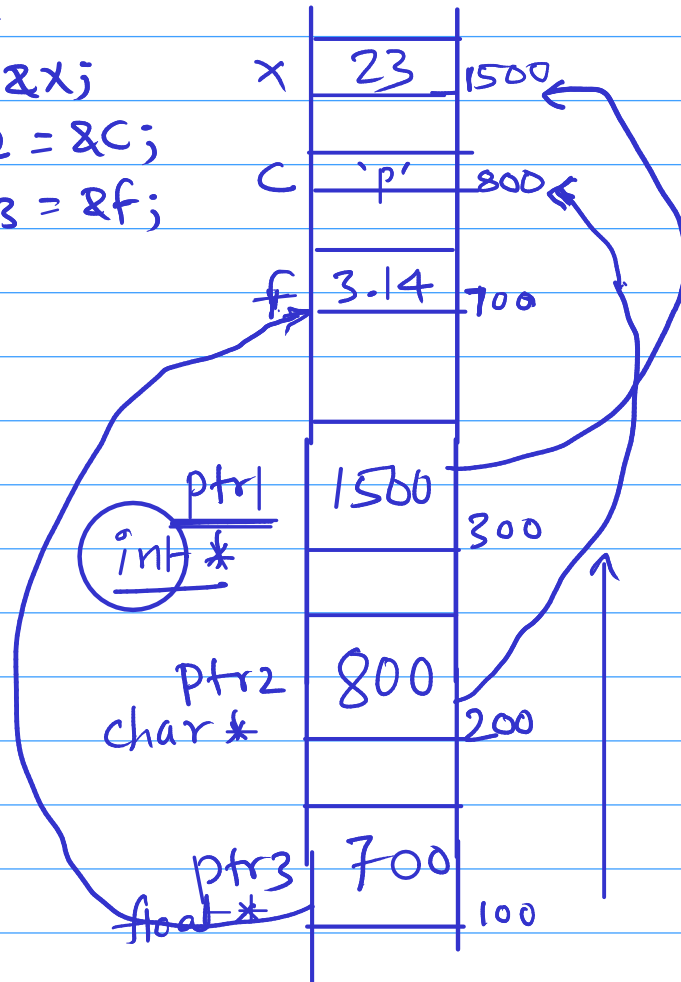
```
float *ptr3 = &f;
```

~~X~~ int *ptr = &23;

→ Address of can not be applied on const-

~~X~~ *(234) = 6;

↳ Dereferencing operator can not be applied on constant.



```
printf("%d", *ptr1); // 23
```

```
printf("%c", *ptr2); // p
```

```
printf("%f", *ptr3); // 3.140000
```

```
int x=100;  
int *ptr;
```

```
ptr = &x;
```

```
int y=200;
```

```
→ ptr = &y;
```

```
(200) printf("%d", *ptr);
```

